WanderMate: Social Mobile Travel Planner

Task 4

Prototype Implementation & Architecture Analysis

Contents

1 Introduction **2**

2 Prototype Implementation **2**
- 2.1 Scope of the Prototype . . . 2
- 2.2 Implemented Feature Pipeline . . . 2
- 2.3 Key Implementation Components . . . 7
- 2.4 Prototype Feature Summary . . . 8

3 Architecture Analysis **9**
- 3.1 Architectures Under Comparison . . . 10
  - 3.1.1 Architecture A — Implemented: Facade + API Gateway (Server-Proxied) . . . 10
  - 3.1.2 Architecture B — Alternative: Direct Client-to-API (No Back-end Proxy) . . . 10
- 3.2 Qualitative Comparison . . . 10
- 3.3 Quantitative Evaluation: NFR-11 (Secret Protection) . . . 12
  - 3.3.1 Threat Model: API Key Extraction from Mobile Bundle . . . 12
- 3.4 Quantitative Evaluation: NFR-08 (Graceful Degradation) . . . 13
  - 3.4.1 Failure Scenario: Google Places API Unavailable . . . 13
- 3.5 Trade-off Summary . . . 14

4 NFR Quantification **15**
- 4.1 NFR-01: API Response Time (Performance) . . . 15
- 4.2 NFR-06: API Quota Consumption (Scalability) . . . 15

5 Conclusion **16**

1. Introduction

This document presents the Task 4 deliverable for WanderMate, consisting of two parts:

1. **Prototype Implementation** — a description of the working prototype that demonstrates one end-to-end non-trivial functionality: the **Explore & POI Discovery → Add to Itinerary → Route Optimisation** pipeline. This pipeline exercises the Facade + Strategy pattern (ADR-003), the API Gateway with secret proxying (ADR-002), real-time collaboration via Firebase (ADR-001), the Observer pattern, multi-layer caching, and offline-first persistence (ADR-004).
2. **Architecture Analysis** — a qualitative and quantitative comparison of the implemented architecture (**Facade + API Gateway** pattern) against an alternative architecture (**Direct Client-to-API**), evaluating trade-offs across two non-functional requirements: NFR-11 (Secret Protection) and NFR-08 (Reliability / Graceful Degradation).

> **Notation.** Requirement identifiers (NFR-06, FR-05) refer to the requirements catalogue in the Task 1 deliverable. Code references use `monospace` font and point to files in the WanderMate repository at `https://github.com/arnav4124/WanderMate`.

2. Prototype Implementation

2.1 Scope of the Prototype

The WanderMate prototype implements a fully functional, end-to-end **contextual POI discovery and itinerary management pipeline**: from authenticated user login, through trip creation, location-aware POI search with interactive mapping, to itinerary stop management with real-time route optimisation and collaborative sync. This non-trivial pipeline traverses all six architectural subsystems defined in Task 1 and exercises all four ADRs from Task 2.

> **End-to-end functionality demonstrated:** A user creates a trip, opens the Explore screen, searches for POIs near their destination (contextual, category-based, or text-based), views results on an interactive Leaflet map, adds selected POIs to their day-by-day itinerary, and the system automatically computes an optimised multi-stop route via OpenRouteService. Collaborators see changes in real time via Firebase.

2.2 Implemented Feature Pipeline

The following table traces the end-to-end feature pipeline through the system, mapping each step to the architectural components, patterns, and requirements it exercises.

#	User Action	System Behaviour	Patterns Used	Reqs Satisfied
1	Sign in (Email or Google OAuth)	Firebase Authentication verifies credentials; `authStore` registers `onAuthStateChanged` listener; JWT token is stored for all subsequent API calls via the Axios interceptor in `services/api.ts`.	Observer, Singleton	FR-22, FR-23
2	Create a new trip	User provides name, destination, date range. `TripController.createTrip` generates the `days[]` array and persists to MongoDB. The `TripObserver` EventEmitter writes a creation event to Firebase RTDB at `trips/{tripId}`.	Observer, Repository	FR-01
3	Open Explore screen	The map auto-centres on the trip's destination by geocoding it via `GET /api/poi/geocode?q=destination`. `TravelService.geocode()` queries Nominatim (5 000 ms timeout), falling back to Google Places on failure. The result is used to initialise the Leaflet WebView map centre.	Facade, Strategy	FR-05, FR-09

#	User Action	System Behaviour	Patterns Used	Reqs Satisfied
4	Search for POIs (text or category)	Client calls `GET /api/poi/search?q=&lat=&l` through the API Gateway. The gateway's middleware stack applies: `helmet()` → `generalLimiter` → `authenticate` (JWT) → `externalApiLimiter` → route handler. The handler delegates to `TravelService.searchPOI()` which checks `node-cache`, calls Google Places (primary), falls back to ORS POI on failure, sorts by Haversine distance, caps at 15 results, caches for 30 minutes, and returns normalised POIs.	Facade, Strategy, Cache-Aside, API Gateway	FR-05, FR-06, FR-07, NFR-01, NFR-06, NFR-08, NFR-11
5	View results on interactive map	POIs render as colour-coded markers (13 categories) on the Leaflet WebView with numbered borders. The Leaflet instance communicates with React Native via `postMessage`/`onMessage` bridge. Map pan/zoom updates are relayed as `REGION_CHANGE` events. A "Search this area" button re-queries the new viewport centre.	Adapter (WebView bridge)	FR-09, NFR-13

#	User Action	System Behaviour	Patterns Used	Reqs Satisfied
6	Add POI to itinerary	User taps "Add to Trip" on a POI card. `tripStore.addStop()` calls `POST /api/trips/:id/days/:dayI` The `TripController` validates ownership/collaborator access, assigns an `order` field, persists to MongoDB, and emits `TRIP_UPDATED` via `TripObserver`. The mutation is pushed to the undo stack (Command pattern). If offline, the mutation is queued via `syncQueue`.	Command, Observer, Repository, Offline-first	FR-02, FR-04, FR-08, FR-20, NFR-07
7	Auto-compute optimised route	When a day has $\geq$ 2 stops, the Explore screen calls `POST /api/routes/optimize` with the stop coordinates. The `TravelService.optimizeRoute()` method delegates to `ORSAdapter.optimizeRoute()`, which uses the VROOM solver (`/optimization` endpoint) to determine optimal visit order, then computes the actual driving route via `/v2/directions`. The resulting GeoJSON polyline is overlaid on the Leaflet map. For $<$ 3 stops, a simple `getRoute()` call is used instead.	Facade, Strategy	FR-10, FR-11

#	User Action	System Behaviour	Patterns Used	Reqs Satisfied
8	Collaborator sees live updates	Collaborator B has an active `onValue` listener on `trips/{tripId}` via `tripStore.subscribeTripUp` When User A adds a stop, the `TripObserver` writes the sanitised trip document to Firebase. Collaborator B's listener fires, compares `updatedAt` timestamps (LWW), and updates local Zustand state. The UI re-renders automatically.	Observer, LWW conflict resolution	FR-20, FR-21, NFR-03
9	Manage stops (reorder, edit, delete)	Reorder: `PUT /api/trips/:id/days/:dayIndex/reorder` with the new stop ID order. Edit: `PUT .../:stopId` updates fields (time, duration, cost, notes); cost edits auto-create or update a linked `Expense` record. Delete: `DELETE .../:stopId` removes the stop and re-indexes remaining stops. All mutations push command objects to the undo stack and emit `TRIP_UPDATED` to Firebase.	Command, Observer, Repository	FR-02, FR-04, FR-12
10	Offline editing & sync	If the device is offline, `fetchTrips()` reads from `localCache` (AsyncStorage, infinite TTL). Write operations catch API errors and enqueue mutations in `syncQueue`. On reconnect, the user triggers "Sync Offline Changes" from the Profile screen; `syncQueue.flush()` replays operations in timestamp order.	Cache-Aside, Offline-first	FR-03, NFR-07

2.3 Key Implementation Components

The prototype is implemented across the following key files:

Backend (Node.js / Express):

File	Role in the Pipeline
`src/index.js`	API Gateway entry point: middleware stack (Helmet, CORS, rate limiter, JWT auth), route registration, global error handler, 404 handler.
`src/controllers/tripCont`	Trip lifecycle management: CRUD, stop add/remove/reorder/update, collaborator management. Emits `TripObserver` events on every mutation. 304 lines.
`src/services/travelService.js`	Façade + Strategy: `searchPOI()`, `searchByCategory()`, `getPlaceDetails()`, `geocode()`, `getRoute()`, `optimizeRoute()`. Haversine proximity sorting, `node-cache` caching, provider fallback. 231 lines.
`src/services/placesAdapt`	Google Places Strategy: Text Search, Nearby Search, Find Place, Place Details. 13-category type mapping. Response normalisation. 189 lines.
`src/services/orsAdapter.js`	ORS Strategy: POI search (`/pois`), route calculation (`/v2/directions`), multi-stop optimisation (`/optimization` via VROOM solver). 12-category ORS mapping. 252 lines.
`src/services/tripObserve`	Observer: Node.js `EventEmitter` decoupling business logic from Firebase writes. Handlers for `TRIP_CREATED`, `TRIP_UPDATED`, `TRIP_DELETED`, `COLLABORATORS_UPDATED`. 54 lines.
`src/routes/poi.js`	POI route handlers: `/search`, `/category/:category`, `/geocode`, `/details/:placeId`. All protected by `externalApiLimiter`. 94 lines.
`src/routes/routes.js`	Route endpoints: `POST /calculate`, `POST /optimize`. Graceful fallback (returns null instead of error). 53 lines.
`src/middleware/rateLimiter.js`	Dual-tier rate limiting: `generalLimiter` (100 req/15 min) and `externalApiLimiter` (50 req/hour).
`src/middleware/auth.js`	Firebase JWT verification via `admin.auth().verifyIdToken()`. Attaches decoded user to `req.user`.

Frontend (React Native / Expo):

File	Role in the Pipeline
`app/(tabs)/explore2.tsx`	Primary Explore screen: unified POI search + interactive Leaflet map + itinerary bottom sheet panel. Handles search, category browsing, map interaction, stop CRUD, route computation, day management. 413 lines.
`stores/tripStore.ts`	Zustand store: trip CRUD, stop management, undo/redo stacks (Command pattern), Firebase `onValue` subscription (Observer), offline cache reads, and sync queue writes. 315 lines.
`stores/budgetStore.ts`	Zustand store: expense CRUD, budget summary fetch, Firebase budget subscription for real-time sync.
`services/syncQueue.ts`	Offline-first infrastructure: `localCache` (AsyncStorage-backed cache with configurable TTL) and `syncQueue` (persistent mutation queue with timestamp-ordered flush). 107 lines.
`services/api.ts`	Axios instance with base URL configuration and automatic Firebase JWT injection via request interceptor.

2.4 Prototype Feature Summary

The prototype implements the following features as part of the end-to-end pipeline:

#	Feature	Description	Requirements
1	Authentication (Email + Google)	Firebase Email/Password and Google Sign-In with persistent sessions via AsyncStorage.	FR-22, FR-23
2	Trip Creation & Management	Create trips with name, destination, date range. Auto-generated day arrays. Full CRUD with owner/collaborator authorisation.	FR-01, FR-02
3	Contextual POI Search	Text-based and category-based search (13 categories) with Haversine proximity sorting and result capping. Dual-provider fallback.	FR-05, FR-06, FR-07

#	Feature	Description	Requirements
4	Interactive Map (Leaflet)	Embedded Leaflet 1.9.4 in WebView with numbered/colour-coded markers, bidirectional JS bridge, viewport tracking, and "Search this area" re-query.	FR-09, FR-11
5	Itinerary Stop Management	Add, remove, reorder, and edit stops with inline cost, duration, notes, and arrival time. Cost edits create linked expenses.	FR-02, FR-08, FR-12
6	Route Optimisation	ORS VROOM solver for optimal multi-stop visit order. GeoJSON polyline overlay on map. Automatic recomputation on stop changes.	FR-10
7	Real-time Collaboration	Firebase RTDB-based Observer pattern. `TripObserver` EventEmitter on backend. Last-write-wins conflict resolution.	FR-20, FR-21
8	Undo/Redo	Command pattern with `undoStack`/`redoStack` in `tripStore`. Supports add, remove, reorder, update operations.	FR-04
9	Offline Support	AsyncStorage-backed `localCache` for reads, `syncQueue` for queued writes, manual flush from Profile screen.	FR-03, NFR-07
10	Destination Geocoding	Nominatim (OSM) with 5 000 ms timeout, Google Places Text Search fallback. Auto-centres map on trip destination.	FR-05

3. Architecture Analysis

This section compares WanderMate's implemented architecture against an alternative architecture. We evaluate both approaches across two non-functional requirements, discuss trade-offs, and provide quantitative evidence for the architectural choices.

3.1 Architectures Under Comparison

3.1.1 Architecture A — Implemented: Facade + API Gateway (Server-Proxied)

WanderMate uses a **backend API Gateway** (Node.js/Express) that proxies all external API calls. The mobile client never directly contacts Google Places, OpenRouteService, or Nominatim. All API keys reside as environment variables on the server. The **TravelService Facade** encapsulates multiple provider adapters behind a single interface, centralising caching, fallback orchestration, proximity sorting, and quota control.

Key structural properties:

- All external API calls are routed: *Client* → *API Gateway* → *External API*
- API keys exist only in server-side `.env`; zero secrets in the mobile bundle
- The Facade provides a common normalised POI interface regardless of provider
- Provider fallback (Google → ORS → empty) is invisible to clients
- `node-cache` caching sits inside the Facade (30 min search / 24 hr details / 1 hr routes)
- Rate limiting (`express-rate-limit`) is enforced at the gateway
- Total middleware chain per request: Helmet → CORS → JSON parser → General rate limit → JWT auth → (external rate limit on POI/route endpoints) → route handler

3.1.2 Architecture B — Alternative: Direct Client-to-API (No Backend Proxy)

In this alternative, the React Native mobile client calls external APIs *directly*. API keys are embedded in the client bundle (or fetched from a lightweight config endpoint). Each screen component is responsible for its own error handling, caching, and response normalisation. There is no backend proxy layer for external API calls; the backend serves only MongoDB CRUD operations and Firebase auth.

Key structural properties:

- External API calls are routed: *Client* → *External API* (no intermediate hop)
- API keys are embedded in the mobile app bundle or environment config
- Each screen/component normalises API responses inline
- No centralised fallback: each call site implements its own error handling
- Caching is done client-side only (AsyncStorage or in-memory)
- No server-side rate limiting on external API calls

3.2 Qualitative Comparison

Quality Attribute	Architecture A: Facade + Gateway	Architecture B: Direct Client-to-API
Security (NFR-11)	API keys never leave the server. Impossible to extract from mobile bundle. Firebase JWT + per-route authorisation enforced centrally.	API keys embedded in JavaScript bundle. Extractable via `apktool`/`jadx`. No centralised auth enforcement for external calls.
Reliability (NFR-08)	Centralised fallback chain in Facade: Google → ORS → empty. One implementation, zero duplication. Provider failures invisible to client.	Each call site must implement its own error handling and fallback. Risk of inconsistent degradation. Provider failure may surface as blank screens in some views.
Performance / Latency	Adds one extra network hop (client → gateway → API). Mitigated by server-side `node-cache`: cached responses skip the external API entirely.	One fewer network hop (client → API directly). Lower latency on cache misses. No server-side cache sharing across users.
Quota Control (NFR-06)	Server-side `node-cache` deduplicates identical requests across all users. `externalApiLimiter` enforces 50 req/hr per IP. Centralised quota management.	No cross-user cache sharing. Each device independently consumes quota. No server-side rate limiting. Quota exhaustion risk grows linearly with user count.
Modifiability (NFR-15)	Swapping a provider requires changes in one adapter file + one line in `TravelService` constructor. Validated by disabling OverpassAdapter: zero changes to routes, stores, or UI.	Provider change requires modifying every screen/component that calls the API. Response normalisation logic scattered across the codebase.

Quality Attribute	Architecture A: Facade + Gateway	Architecture B: Direct Client-to-API
Operational Complexity	Requires deploying and maintaining a backend service. Single point of failure: if the gateway crashes, all external API access is lost.	No backend dependency for external API calls. The client is self-sufficient. Simpler deployment for POI features.
Offline Support (NFR-07)	Both architectures can implement client-side caching via AsyncStorage. No material difference for offline reads.	Same.

3.3 Quantitative Evaluation: NFR-11 (Secret Protection)

NFR-11 states: *"Third-party API keys shall never be embedded in the mobile app bundle. All calls to external APIs shall be proxied through the backend API Gateway."*

This NFR is binary — either secrets are protected, or they are not. We evaluate both architectures against a concrete threat model.

3.3.1 Threat Model: API Key Extraction from Mobile Bundle

React Native applications compile to JavaScript bundles packaged inside APK (Android) or IPA (iOS) files. These bundles can be extracted and inspected using freely available tools (`apktool` for Android, `class-dump` for iOS). Any string constant — including API keys — present in the bundle is effectively a public value.

Metric	Arch. A (Gateway)	Arch. B (Direct)	Assessment
API keys in client bundle	**0**	**3** (Google Places, ORS, Nominatim User-Agent)	A is strictly superior
Extraction effort	N/A (no keys to extract)	$<$ 5 minutes with `apktool -d app.apk`	B is trivially compromised
Key rotation impact	Change `.env` and restart server; zero client changes	Requires rebuilding and redeploying the mobile app to all users	A has zero-downtime rotation

Metric	Arch. A (Gateway)	Arch. B (Direct)	Assessment
Abuse potential	Attacker cannot access keys; abuse requires attacking the gateway itself	Attacker uses extracted key to make arbitrary API calls, incurring cost to the project team	B creates direct financial liability

> **Verdict for NFR-11.** Architecture A provides **complete secret protection** with zero API keys in the client bundle. Architecture B fundamentally *cannot* satisfy NFR-11; embedding keys in the mobile bundle is the architectural equivalent of publishing them publicly. The Gateway pattern is the only viable approach for this requirement.

3.4 Quantitative Evaluation: NFR-08 (Graceful Degradation)

NFR-08 states: *"If the Google Places API is unavailable, POI search shall degrade gracefully to ORS POI results; no user-facing crash or blank screen shall occur."*

We compare how each architecture handles the scenario where Google Places returns an error (quota exceeded, network timeout, or API outage).

3.4.1 Failure Scenario: Google Places API Unavailable

Metric	Arch. A (Gateway)	Arch. B (Direct)
Fallback implementation	Single `try/catch` in `TravelService.searchPOI()`: Google → ORS → empty array. **1 code location**, 6 lines.	Each screen/component calling Google Places must independently implement fallback logic. Minimum **3 code locations** (Explore search, category browse, geocode), each with its own error handling.
Code duplication	Zero. Fallback is centralised in the Facade.	Fallback logic duplicated per call site. Inconsistencies are likely over time as different developers maintain different screens.
User experience on failure	Transparent. ORS results are returned with the same normalised shape. UI renders them identically. The user sees fewer results but no error.	Depends on per-screen implementation. Some screens may show error alerts, others may show blank results, creating an inconsistent UX.

Metric	Arch. A (Gateway)	Arch. B (Direct)
Failure detection latency	Bounded by `axios` timeout (10 s for Places). Server logs capture all failures centrally for monitoring.	Bounded by client-side timeout. Failures are logged only on the device; no central monitoring.
Cascading failure risk	Low. If Google fails, the Facade falls to ORS. If ORS also fails, an empty array is returned. The API Gateway itself remains operational.	Higher. Without centralised coordination, the client may retry aggressively against a failing API, exacerbating quota or timeout issues.

Verdict for NFR-08. Architecture A's centralised Facade provides **guaranteed, consistent graceful degradation** with a single fallback chain. Architecture B *can* implement degradation, but at the cost of code duplication and consistency risk, making it structurally less reliable for this requirement.

3.5 Trade-off Summary

Trade-off Dimension	Architecture A Wins	Architecture B Wins
Security (NFR-11)	**Yes** — zero secrets in client	No
Reliability (NFR-08)	**Yes** — centralised fallback	Partial (depends on per-site implementation)
Quota Control (NFR-06)	**Yes** — server-side cache sharing + rate limiting	No
Modifiability (NFR-15)	**Yes** — single adapter swap, validated empirically	No
Raw Latency	No — extra network hop adds ∼20–80 ms	**Yes** — one fewer hop
Operational Simplicity	No — requires backend deployment	**Yes** — no server for external APIs
Offline Support (NFR-07)	Neutral	Neutral

Overall assessment. Architecture A (Facade + API Gateway) is the correct choice for WanderMate. The security guarantee of NFR-11 alone is non-negotiable — embedding API keys in a mobile bundle is an industry-recognised anti-pattern. The additional benefits in reliability, quota control, and modifiability far outweigh the marginal latency overhead and operational complexity of maintaining a backend service.

> Architecture B would be appropriate only for applications where: (a) all external APIs are free and keyless, (b) no centralised quota management is needed, and (c) provider stability is high enough that fallback is unnecessary. None of these conditions hold for WanderMate.

4. NFR Quantification

To provide concrete quantitative evidence for the architectural choices, we measure the following non-functional requirements under controlled conditions.

4.1 NFR-01: API Response Time (Performance)

NFR-01 requires POI search results to be returned within 2 seconds on a 4G connection.

Scenario	Arch. A (Gateway)	Arch. B (Direct)	Target
Cold POI search (cache miss)	*TBD*	*TBD*	$< 2\,\mathrm{s}$
Warm POI search (cache hit)	*TBD*	*TBD*	$< 2\,\mathrm{s}$
Route optimisation (5 stops)	*TBD*	*TBD*	$< 2\,\mathrm{s}$
Geocode (Nominatim)	*TBD*	*TBD*	$< 2\,\mathrm{s}$

> **Note.** Quantitative latency measurements will be populated after benchmark testing against both architectures. The expectation is that Architecture A will show slightly higher cold-miss latency due to the gateway hop, but **significantly lower warm latency** due to server-side cache sharing (cache hits skip the external API entirely and return in microseconds from `node-cache`).

4.2 NFR-06: API Quota Consumption (Scalability)

NFR-06 requires the system to operate within free-tier quotas: Google Places $\leq 5\,000$ req/month, ORS $\leq 2\,000$ req/day.

Scenario (10-user session)	Arch. A (Gateway)	Arch. B (Direct)	Quota Limit
Google Places calls (1 hour)	*TBD*	*TBD*	5000/month
ORS calls (1 hour)	*TBD*	*TBD*	2000/day

Scenario (10-user session)	Arch. A (Gateway)	Arch. B (Direct)	Quota Limit
Cache hit ratio	*TBD*	*N/A (no server cache)*	–

> **Note.** The expected result is that Architecture A's server-side `node-cache` will dramatically reduce outbound API calls by serving identical queries from cache. In Architecture B, each client maintains its own cache, so the same query from 10 different users generates 10 separate API calls.

5. Conclusion

This document has presented the Task 4 deliverable for WanderMate:

Prototype Implementation. A fully functional end-to-end pipeline has been implemented, spanning the complete lifecycle from authenticated user sign-in through contextual POI discovery, interactive mapping, itinerary management with real-time route optimisation, to collaborative editing and offline persistence. This pipeline exercises all six subsystems, all four ADRs, and all five tactics defined in Tasks 1–3. The prototype is not a minimal skeleton — it is a working application with 13 POI categories, Haversine proximity sorting, Leaflet WebView integration, VROOM-based route optimisation, undo/redo command stacks, Firebase real-time collaboration, and offline-first caching.

Architecture Analysis. A systematic comparison between the implemented **Facade + API Gateway** architecture and a **Direct Client-to-API** alternative was conducted across two non-functional requirements:

- **NFR-11 (Secret Protection):** Architecture A provides complete secret protection with zero API keys in the client bundle. Architecture B fundamentally cannot satisfy this requirement.
- **NFR-08 (Graceful Degradation):** Architecture A's centralised Facade provides guaranteed, consistent fallback with a single implementation point. Architecture B can implement degradation but at the cost of code duplication and consistency risk.

The Facade + API Gateway architecture introduces marginal latency overhead and operational complexity but delivers non-negotiable security, reliability, quota control, and modifiability benefits that are essential for a mobile application consuming quota-limited third-party APIs.